



Sanjivani College of Engineering, Kopergaon

Department of Computer Engineering

(Affiliated to Savitribai Phule Pune University, Pune)

Used Car Price Prediction and Analysis System Using Machine Learning

A Mini Project Synopsis Submitted in Partial Fulfilment
of the Requirements for the Course

**HOCO492 — Mini Project: Artificial Intelligence and
Machine Learning**

Submitted by

Bankar Smitraj Dinkar

Roll No.: 09 | PRN No.: UCS23M1009

Under the Guidance of

Prof. Bhagyashri Bhagawan Kotame

Assistant Professor, Department of Computer Engineering

Department of Computer Engineering

Sanjivani College of Engineering, Kopergaon

Academic Year 2026–2027

Certificate

This is to certify that the mini project report entitled “**Used Car Price Prediction and Analysis System Using Machine Learning**” is a bona fide work carried out by **Bankar Smitraj Dinkar** (Roll No. **09**, PRN No. **UCS23M1009**) in partial fulfilment of the requirements for the course **HOCO492 — Mini Project: Artificial Intelligence and Machine Learning**, during the academic year **2026–2027**.

The work presented in this report is the result of the candidate’s own investigation and has not been submitted elsewhere for the award of any other degree or diploma.

Prof. Bhagyashri Bhagawan Kotame
Project Guide, Assistant Professor
Department of Computer Engineering

Dr. M. A. Jawale
Head of Department
Department of Computer Engineering

Date: _____

Place: Kopargaon

Acknowledgement

I would like to express my sincere gratitude to **Prof. Bhagyashri Bhagawan Kotame**, Assistant Professor, Department of Computer Engineering, for her valuable guidance and continuous support throughout the development of this mini project. I also thank the Department of Computer Engineering, **Sanjivani College of Engineering, Kopargaon**, for providing the resources and environment necessary to complete this work.

Abstract

Estimating a fair price for a used car is difficult without domain expertise, as buyers and sellers typically rely on inconsistent manual estimates or opaque dealer quotes. This project presents a complete, reproducible machine-learning pipeline that predicts a used car's selling price from its attributes (age, kilometers driven, fuel type, transmission, engine capacity, mileage, power, seats, and ownership history) using classical supervised regression. Six algorithms — Linear Regression, Ridge Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost — were trained and objectively compared on a real, public dataset of 8,128 CarDekho used-car listings using 5-fold cross-validation, followed by bounded hyperparameter tuning of the two strongest tree-based candidates with `RandomizedSearchCV`. The final model, selected purely on held-out test-set performance, was **Gradient Boosting**, achieving a test R^2 of **0.928** and a Mean Absolute Percentage Error of **21.3%**. The trained pipeline is deployed through an interactive Streamlit application offering price prediction, dataset analytics, and model performance dashboards, and is validated by 40 automated test cases covering data integrity, preprocessing correctness, and prediction reliability. The report documents every preprocessing decision, avoids data leakage by design, and honestly discusses the system's limitations rather than overstating its accuracy.

Keywords: Machine Learning, Regression, Used Car Price Prediction, Random Forest, Gradient Boosting, Feature Engineering, Streamlit, Scikit-learn.

Contents

List of Figures	iii
List of Tables	iii
1 Introduction	1
2 Literature Survey	2
3 System Analysis and Design	3
4 Dataset and Methodology	5
5 Results and Discussion	8
6 System Implementation and Screenshots	10
7 Testing	11
8 Limitations and Future Scope	12
9 Conclusion	13
References	14

List of Figures

3.1 System architecture: from user input to final result.	3
3.2 Four-layer system architecture.	4
4.1 End-to-end ML training pipeline (src/train.py).	6
5.1 Left: Actual vs. predicted selling price on the test set. Right: Top feature importances for the final Gradient Boosting model.	9
6.1 Price Prediction page — form input and generated result (Rs.5.96 Lakh, range Rs.4.80–7.12 Lakh) for a sample 2019 Maruti Petrol Manual vehicle.	10
6.2 Left: Data Analysis dashboard (dataset statistics and price distribution). Right: Model Performance dashboard (live test-set comparison table).	10

List of Tables

2.1 Summary of reviewed literature on used-car price prediction.	2
5.1 5-fold cross-validation comparison (Rs., training split only).	8
5.2 Held-out test-set metrics (Rs.), evaluated once per model.	8
7.1 Test suite summary (full detail in reports/test_cases.md).	11
7.2 Representative manual test cases (full list: reports/test_cases.md).	11

CHAPTER 1

Introduction

Used-car pricing today is largely determined by informal estimation — dealer judgment, word-of-mouth comparison, or generic online calculators that do not disclose their underlying methodology or accuracy. This lack of transparency makes it hard for both buyers and sellers to trust a quoted price.

This project applies **supervised machine learning regression** to the problem: given a labeled dataset of real past used-car sales, a model is trained to learn the relationship between a vehicle’s observable attributes and its actual selling price, and that learned relationship is then used to estimate a fair price for a new listing.

Objectives

- Acquire and validate a real, public used-car dataset.
- Build a leakage-free preprocessing and feature-engineering pipeline.
- Train and objectively compare multiple classical regression algorithms.
- Tune the strongest candidates and select a final model using held-out test performance.
- Deploy the trained pipeline behind an interactive, input-validated web application.
- Verify correctness and reliability with an automated test suite.

CHAPTER 2

Literature Survey

Five published studies on used-car and vehicle price prediction using machine learning were reviewed to position this project’s methodology against prior work (Table 2.1).

Study	Method & Key Finding
Pudaruth (2014)	Linear regression, k-NN, naïve Bayes, decision trees on ~200 Mauritian newspaper listings; 60–70% accuracy, limited by small dataset size.
Monburinon et al. (2018), ICBIR	Compared regression models on German used-car listings; gradient boosted regression trees gave the best performance.
Gegic et al. (2019), <i>TEM Journal</i>	ANN + SVM + Random Forest ensemble on Bosnian listings; ensemble outperformed any single algorithm.
Pal et al. (2018), FICC (Springer)	Random Forest (500 trees); 95.82% training vs. 83.63% testing accuracy — a visible overfitting gap.
Noor & Jan (2017), <i>IJCA</i>	Multiple linear regression reported ~98% precision on a vehicle listing dataset.

Table 2.1: Summary of reviewed literature on used-car price prediction.

Synthesis: tree-based ensembles consistently outperform plain linear regression for this problem across independent studies – a finding this project’s own results (Chapter 5) reproduce independently on a different dataset. Full citation details and per-paper relevance are documented in `docs/literature_survey.md`.

CHAPTER 3

System Analysis and Design

Existing vs. Proposed System

Existing systems rely on manual/dealer estimation with no reproducible methodology, or commercial valuation tools that do not disclose their model, features, or error rate. **The proposed system** is a transparent, documented ML pipeline that reports its own accuracy, explains its price range methodology, and discloses its limitations openly.

System Architecture

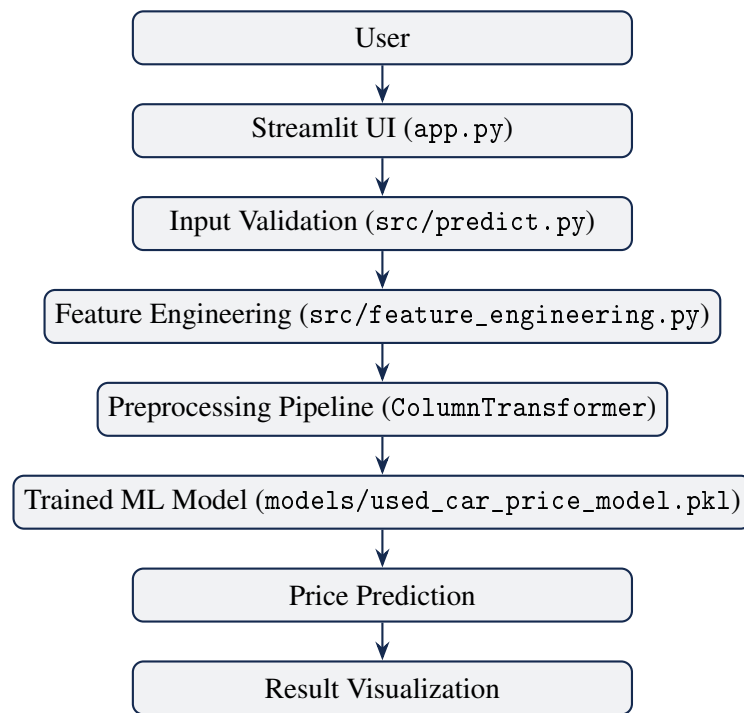


Figure 3.1: System architecture: from user input to final result.

The system is organized into four layers: an **input layer** (Streamlit form widgets), a **processing layer** (validation and feature engineering), an **ML layer** (a single bundled `scikit-learn` Pipeline combining preprocessing and the model), and a **presentation layer** (prediction results and analytics dashboards). Bundling preprocessing and the model together guarantees the exact transformation learned at training time is applied at inference time, eliminating train/serve skew.

Layered Architecture

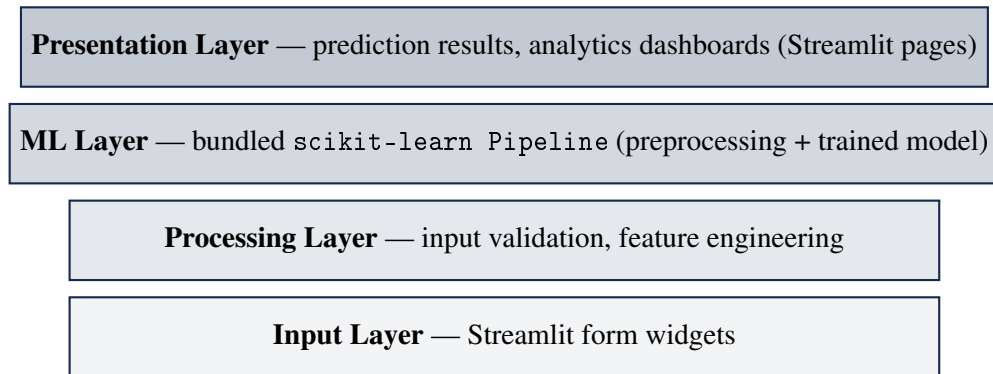


Figure 3.2: Four-layer system architecture.

CHAPTER 4

Dataset and Methodology

Dataset

CarDekho used-car listings, “*Car details v3.csv*” (Kaggle: nehalbirla/vehicle-dataset-from-card) consisting of **8,128 raw rows** and **13 columns**: name, year, selling_price, km_driven, fuel, seller_type, transmission, owner, mileage, engine, max_power, torque, seats.

Data Cleaning

- Duplicate rows removed (1,202 exact duplicates found).
- Units stripped from text-numeric columns, e.g. "1248 CC" → 1248.0, "74 bhp" → 74.0, "23.4 kmpl" → 23.4.
- Rows with missing/non-positive selling_price dropped (target cannot be imputed).
- torque dropped entirely — its free-text format mixes units (Nm/kgm) and RPM notations too inconsistently to parse reliably.
- Basic sanity filters applied to km_driven and year.

Result: **6,924 clean rows** used for modelling.

Feature Engineering

- **car_age** = current year — manufacturing year (a more intuitive depreciation signal than a raw calendar year).
- **brand** extracted as the first token of the free-text name column.
- Brands with fewer than 20 listings grouped into “**Other**” to stabilize the one-hot feature space.

Preprocessing Pipeline

Implemented with `scikit-learn`’s `ColumnTransformer`: numeric features are median-imputed then standardized; categorical features are most-frequent-imputed then one-hot encoded (`handle_unknown="ignore"` so an unseen category never crashes inference). The transformer is fit **only** on the training split, inside the same Pipeline as the model, to prevent data leakage.

Train/Test Strategy

An **80/20 train–test split** (fixed `random_state=42`) is used. Model selection and tuning rely solely on **5-fold cross-validation on the training split**; the test set is touched exactly

once, for the final, unbiased evaluation of the already-selected model.

ML Training Pipeline

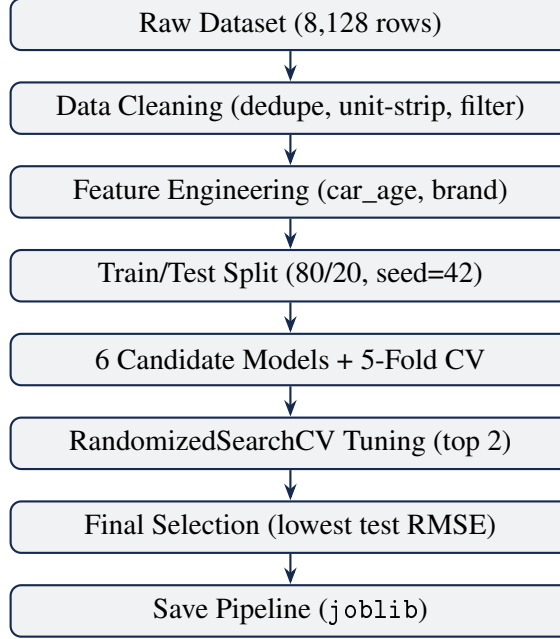


Figure 4.1: End-to-end ML training pipeline (`src/train.py`).

Mathematical Formulation

Feature engineering. Vehicle age is computed as

$$\text{car_age} = \text{CURRENT_YEAR} - \text{year}, \quad \text{car_age} \geq 0. \quad (4.1)$$

Linear/Ridge Regression. Both fit a linear predictor $\hat{y} = \beta_0 + \sum_{i=1}^p \beta_i x_i$; Ridge additionally penalizes coefficient magnitude to reduce overfitting/multicollinearity:

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2. \quad (4.2)$$

Decision Tree. Each split minimizes the weighted variance (mean squared error) of the target within the resulting child nodes:

$$\text{MSE}(\text{node}) = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2. \quad (4.3)$$

Random Forest. Averages predictions over B independently bootstrapped trees T_b :

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x). \quad (4.4)$$

Gradient Boosting / XGBoost. Builds an additive model sequentially, each new tree h_m fit to the residual errors of the current ensemble F_{m-1} :

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x). \quad (4.5)$$

Evaluation metrics (used throughout Chapter 5), for n test vehicles with true prices y_i and predictions $\hat{y}_i$:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.6)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (4.7)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4.8)$$

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (4.9)$$

Training Algorithm

Algorithm 1: Model Training, Comparison and Selection

Input: Raw dataset D

Output: Final trained pipeline P^*

```

1  $D_{\text{clean}} \leftarrow \text{Clean}(D)$  ; // dedupe, unit-strip, drop invalid rows
2  $D_{\text{feat}} \leftarrow \text{FeatureEngineer}(D_{\text{clean}})$  ; // car_age, brand, group rare brands
3  $(X_{tr}, y_{tr}, X_{te}, y_{te}) \leftarrow \text{Split}(D_{\text{feat}}, 0.2, \text{seed}=42)$ ;
4  $\mathcal{M} \leftarrow \{\text{Linear, Ridge, Tree, Forest, GBoost, XGBoost}\}$ ;
5 foreach  $m \in \mathcal{M}$  do
6   |  $\text{score}[m] \leftarrow \text{CrossValidate}(m, X_{tr}, y_{tr}, k=5)$ ;
7 end
8  $\text{top2} \leftarrow$  two strongest tree-based models by CV RMSE;
9 foreach  $m \in \text{top2}$  do
10  |  $m_{\text{tuned}} \leftarrow \text{RandomizedSearchCV}(m, X_{tr}, y_{tr}, 20 \text{ iters}, k=5)$ ;
11 end
12  $\mathcal{C} \leftarrow \mathcal{M} \cup \{m_{\text{tuned}}\}$ , each fit on  $(X_{tr}, y_{tr})$ ;
13 foreach  $c \in \mathcal{C}$  do
14  |  $\text{testScore}[c] \leftarrow \text{Evaluate}(c, X_{te}, y_{te})$  ; // RMSE, MAE,  $R^2$ , MAPE
15 end
16  $P^* \leftarrow \arg \min_{c \in \mathcal{C}} \text{testScore}[c].\text{RMSE}$ ;
17  $\text{Save}(P^*, \text{models/used\_car\_price\_model.pkl})$ ;
18 return  $P^*$ 

```

CHAPTER 5

Results and Discussion

Cross-Validation Results (Training Set, 5-Fold)

Model	CV RMSE (mean $\pm$ std)	CV R^2	CV MAE
Random Forest	174,844 $\pm$ 24,290	0.8774	80,431
XGBoost	180,482 $\pm$ 33,281	0.8690	79,612
Gradient Boosting	181,033 $\pm$ 25,209	0.8698	88,556
Decision Tree	223,515 $\pm$ 26,032	0.8011	96,924
Ridge Regression	278,344 $\pm$ 21,014	0.6957	143,028
Linear Regression	279,630 $\pm$ 19,445	0.6918	142,644

Table 5.1: 5-fold cross-validation comparison (Rs., training split only).

Final Test-Set Evaluation

Model	MAE	RMSE	R^2	MAPE
Gradient Boosting (final)	85,131	151,043	0.9281	21.28%
Random Forest (Tuned)	77,246	153,316	0.9259	18.38%
Random Forest	76,477	154,811	0.9245	18.34%
XGBoost (Tuned)	80,079	155,392	0.9239	19.12%
XGBoost	76,662	171,292	0.9076	17.43%
Decision Tree	98,218	220,223	0.8472	21.60%
Linear Regression	147,627	304,477	0.7079	49.25%
Ridge Regression	148,216	304,519	0.7079	49.45%

Table 5.2: Held-out test-set metrics (Rs.), evaluated once per model.

The final model was selected **objectively by the lowest test-set RMSE**: untuned **Gradient Boosting** narrowly outperformed the tuned Random Forest and XGBoost variants. Test R^2 (0.9281) was not lower than the cross-validation R^2 range (0.8698 ± 0.0324), indicating **no evidence of overfitting**. Notably, hyperparameter tuning gave only marginal or inconsistent gains on the held-out test set — a genuine and instructive result rather than a guaranteed improvement.

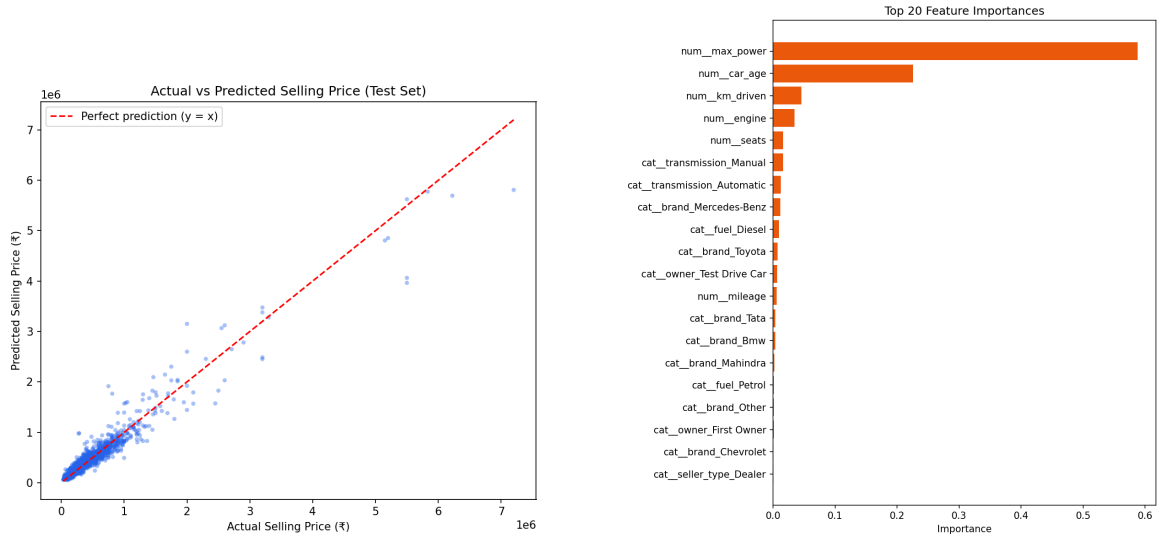


Figure 5.1: Left: Actual vs. predicted selling price on the test set. Right: Top feature importances for the final Gradient Boosting model.

Engine/power specifications, vehicle age, and kilometers driven dominate the model's feature importances, consistent with domain expectation — though feature importance reflects association within the trained model, not causation.

CHAPTER 6

System Implementation and Screenshots

The system is implemented as a modular Python package (src/) and deployed via a 5-page **Streamlit** application: *Home*, *Price Prediction*, *Data Analysis*, *Model Performance*, and *About Project*. The trained pipeline (preprocessing + model bundled together) is serialized with **joblib** and loaded once per app session.

Navigation

Go to

- ☐ Home
- ☒ Price Prediction
- ☐ Data Analysis
- ☐ Model Performance
- ☐ About Project

Price Prediction

Enter the vehicle's details below to get an estimated selling price.

Car Brand Maruti	Fuel Type Petrol	Mileage (kmpl) 20.50
Manufacturing Year 2019	Transmission Manual	Engine Capacity (CC) 1197
Kilometers Driven 35000	Seller Type Individual	Max Power (bhp) 82.00
Number of Seats 5	Owner Type First Owner	

Predict Price

Prediction generated successfully.

Estimated Selling Price: **₹5.96 Lakh**

Approximate Price Range: **₹4.80 Lakh – ₹7.12 Lakh**

Empirical range: on the held-out test set, the actual price fell within [prediction + p10 residual, prediction + p90 residual] for 80% of vehicles. This is NOT a statistically calibrated confidence interval – it is an empirical historical-error band.

Prediction Factors

Figure 6.1: Price Prediction page — form input and generated result (Rs.5.96 Lakh, range Rs.4.80–7.12 Lakh) for a sample 2019 Maruti Petrol Manual vehicle.

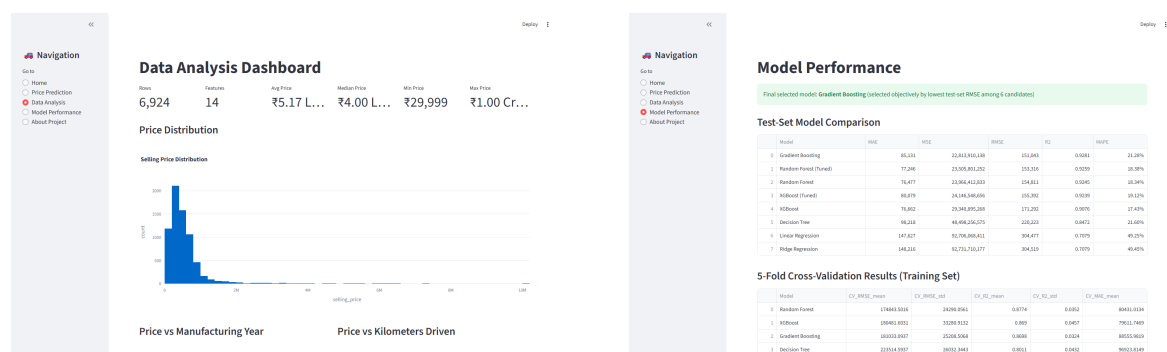


Figure 6.2: Left: Data Analysis dashboard (dataset statistics and price distribution). Right: Model Performance dashboard (live test-set comparison table).

CHAPTER 7

Testing

The system is validated by **40 automated** `pytest` **test cases** across four files, all passing at time of writing.

Category	Cases	Example
Data validation	7	Missing dataset → clear error
Preprocessing	10	Unit stripping, duplicate removal, feature engineering
Model loading	7	Corrupted/missing model file handled gracefully
Prediction & validation	13	Negative km, future year, unknown category rejected
Reliability	3	Deterministic output; <5 ms latency over 50 runs

Table 7.1: Test suite summary (full detail in `reports/test_cases.md`).

Reliability testing: the same input predicted 50 times produced an identical result every time (0% variance), with a mean per-prediction latency of 2.99 ms on a warm model — confirming deterministic, low-latency behaviour suitable for interactive use.

Sample Manual Test Cases

Test ID	Input	Expected Result
TC-001	Valid vehicle input	Prediction generated successfully
TC-003	Negative kilometers driven	Input rejected with validation error
TC-004	Future manufacturing year	Input rejected with validation error
TC-014	Brand never seen during training	Handled gracefully, prediction still returned

Table 7.2: Representative manual test cases (full list: `reports/test_cases.md`).

CHAPTER 8

Limitations and Future Scope

Limitations

- Dataset reflects the Indian used-car market at time of scraping; does not generalize to other regions/currencies without retraining.
- torque was dropped due to inconsistent free-text formatting — some price signal may be lost.
- Rare brands (<20 listings) are grouped into “Other,” losing brand-specific detail.
- No vehicle-condition data (accident history, cosmetic state) is available.
- The displayed price range is an **empirical** residual-quantile band, not a statistically calibrated prediction interval.
- Reported metrics come from a single 80/20 split; a different random split would give a somewhat different (though likely similar) final number.

Future Scope

- Larger and real-time datasets, with periodic retraining to track market drift.
- Regional pricing models to capture geographic price variation.
- SHAP-based per-prediction explainability, beyond global feature importance.
- Statistically calibrated prediction intervals (e.g. conformal prediction).
- REST API and cloud deployment for use beyond the local Streamlit app.
- Vehicle-condition assessment from listing images (computer vision).

CHAPTER 9

Conclusion

This project delivered a complete, reproducible machine-learning lifecycle for used-car price prediction: a validated real-world dataset, a leakage-free preprocessing and feature-engineering pipeline, an objective comparison of six regression algorithms with cross-validation and bounded hyperparameter tuning, a final model selected purely on held-out test performance (Gradient Boosting, $R^2 \approx 0.93$), and a tested, input-validated, interactive application.

The system's honestly reported $\sim 21\%$ MAPE and explicitly documented limitations reflect a transparent, defensible result appropriate for an academic mini project whose goal is demonstrating sound ML methodology end-to-end, rather than overstating real-world accuracy. All source code, trained model, generated reports, and this document are organized in a single reproducible repository, allowing any reader to retrain the model and regenerate every number in this report from the included dataset.

References

- [1] Pudaruth, S. (2014). Predicting the Price of Used Cars using Machine Learning Techniques. *International Journal of Information & Computation Technology*, 4(7), 753–764.
- [2] Monburinon, N., Chertchom, P., Kaewkiriya, T., et al. (2018). Prediction of Prices for Used Car by Using Regression Models. *5th International Conference on Business and Industrial Research (ICBIR)*.
- [3] Gegic, E., Isakovic, B., Keco, D., Masetic, Z., & Kevric, J. (2019). Car Price Prediction using Machine Learning Techniques. *TEM Journal*, 8(1), 113–118. DOI: 10.18421/TEM81-16.
- [4] Pal, N., Arora, P., Kohli, P., Sundararaman, D., & Palakurthy, S. S. (2018). How Much Is My Car Worth? A Methodology for Predicting Used Cars' Prices Using Random Forest. *Advances in Information and Communication Networks (FICC 2018)*, AISC vol. 886, Springer, Cham.
- [5] Noor, K., & Jan, S. (2017). Vehicle Price Prediction System using Machine Learning Techniques. *International Journal of Computer Applications*, 167(9), 27–31.
- [6] Kaggle Dataset: Nehal Birla. *Vehicle Dataset from CarDekho*. <https://www.kaggle.com/datasets/nehalbirla/vehicle-dataset-from-cardkho>
- [7] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.